Study Material — Part 1: RAG, Chunking, Embeddings, Vector Search & Retrieval

Deliverable 4 (Part 1 of several) — Databricks GenAI Engineer Associate

This covers Section 2 (Data Preparation) almost entirely, plus large parts of Sections 3 and 4. If you're new to GenAI, read this part in order — each topic builds on the last.

Topic 1: Retrieval-Augmented Generation (RAG)

What is it?

RAG is a technique where, instead of relying only on what an LLM "knows" from its training data, you **retrieve relevant information from an external knowledge source at query time** and feed that information into the LLM's prompt as context, so it can generate a grounded, up-to-date, and specific answer.

Think of it like an "open-book exam" for the LLM: instead of expecting it to have memorized everything, you hand it the exact pages of the textbook relevant to the question right before it answers.

Why is it used?

LLMs have three core limitations that RAG addresses:

1. **Knowledge cutoff** — an LLM only knows what it was trained on, up to some date. RAG lets it "know" about anything you can retrieve, including today's data.
2. **No access to private/proprietary data** — a general-purpose LLM has never seen your company's internal HR policies, product manuals, or customer records. RAG lets you ground answers in *your* data without retraining the model.
3. **Hallucination** — LLMs can generate plausible-sounding but false information. Giving the model real retrieved source text to answer from significantly reduces (though doesn't eliminate) this risk, and lets you cite sources.

How does it work? (the core pipeline)

1. **Ingest** — collect your source documents (PDFs, web pages, database records, etc.)
2. **Chunk** — split documents into smaller pieces (see Topic 2)
3. **Embed** — convert each chunk into a numerical vector representation (see Topic 3)
4. **Index/Store** — store those vectors in a vector database/index (see Topic 4)
5. **Query time:** embed the user's question the same way, then **retrieve** the most similar chunks from the index (see Topic 5)
6. **Augment** — insert the retrieved chunks into the LLM's prompt as context

7. **Generate** — the LLM produces an answer grounded in that context

Important terminology

- **Retriever** — the component that fetches relevant chunks given a query
- **Grounding** — ensuring the LLM's answer is based on retrieved facts, not just its internal knowledge
- **Context window** — the maximum amount of text (measured in tokens) an LLM can process in one call; retrieved chunks + the question + instructions all must fit inside it
- **Groundedness** (as an evaluation concept) — whether a response is actually supported by the retrieved context

Databricks implementation

On Databricks, a typical RAG stack is:

- **Unity Catalog Volumes** or Delta tables — store raw documents and/or chunked text
- **Mosaic AI Vector Search** — the managed vector database/index service (Topic 4)
- **Foundation Model APIs** — serve the embedding model and the generation LLM
- **MLflow** — log the RAG chain as a model (often a `pyfunc` or LangChain model) with a signature, dependencies, and input examples, then register it to Unity Catalog and deploy via Model Serving
- **Mosaic AI Agent Framework / Agent Evaluation** — build and evaluate the RAG application end-to-end

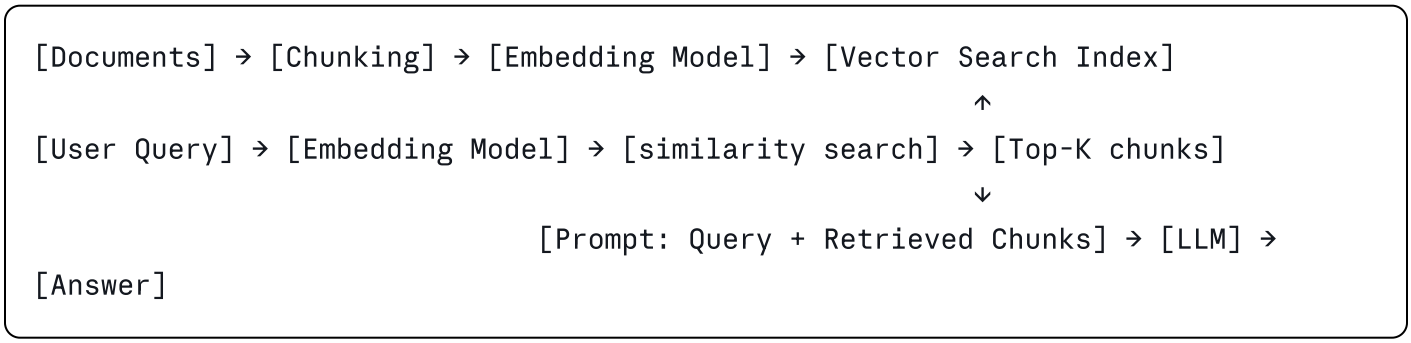
Real-world example

An internal HR chatbot: employees ask "How many vacation days do I get after 3 years?" The system retrieves the relevant paragraph from the company's HR policy PDF and the LLM answers using that exact paragraph, rather than guessing from general HR knowledge it may have seen during training (which would be wrong for *your* company's specific policy).

Simple example

```
User: "What's our return policy for electronics?"
→ System embeds the question
→ Searches vector index of your policy documents
→ Retrieves: "Electronics may be returned within 30 days of purchase with
original receipt..."
→ Prompt sent to LLM: "Using ONLY the following context, answer the question:
[context] + [question]"
→ LLM: "You can return electronics within 30 days of purchase, provided you have
the original receipt."
```

Architecture/workflow diagram (text form)



Important exam points

- RAG does **not** modify the LLM's weights — it changes what's in the *prompt*. This is the key distinction vs. fine-tuning.
- A RAG system should be evaluated **in two separate parts**: retrieval quality (are we finding the right chunks?) and generation quality (given good chunks, does the LLM answer well?). See Q17 in your practice set.
- The "basic RAG application parts list" the exam tests directly: **model flavor, embedding model, retriever, dependencies, input examples, model signature** — know these as the literal checklist for what you configure when logging a RAG chain in MLflow.

Common mistakes/traps

- Assuming RAG "teaches" the model new facts permanently — it doesn't; each query is independent, and nothing is retained after the response unless you separately implement memory.
- Retrieving too much context "just in case," which can dilute relevance, increase cost/latency, and even confuse the LLM (too much irrelevant context can hurt more than help).
- Forgetting that RAG quality is bottlenecked by the *weakest* stage — perfect generation can't save bad retrieval, and perfect retrieval can't save a bad generation prompt.

Comparison with related concepts

	RAG	Fine-tuning
Changes model weights?	No	Yes
Adds new knowledge how?	At query time, via retrieved context	Baked into weights during training
Best for	Fast-changing or proprietary knowledge, source citation needed	Changing style/behavior/format, deep domain adaptation
Cost to update knowledge	Cheap — just update the document store	Expensive — requires retraining

Example exam questions (style)

- "A company wants their chatbot to always cite the exact policy document a customer question was answered from. Should they use RAG or fine-tuning?" → **RAG** (source attribution requires retrieval of real documents, not baked-in weights)
- "Which best describes what RAG changes about how an LLM produces its response?" → It supplies additional context in the prompt at inference time; it does not alter model parameters.

Topic 2: Chunking Strategies

What is it?

Chunking is splitting a large document into smaller, manageable pieces ("chunks") before embedding and indexing them. Because embedding models and LLM context windows have size limits, and because retrieval works best when a chunk is topically focused, you can't just embed an entire 200-page manual as one unit.

Why is it used?

- Embedding models have a maximum input length (their "context length" — see the embedding-model-context-length exam objective specifically).
- Smaller, focused chunks retrieve more precisely — if a chunk mixes 5 unrelated topics, it becomes a "blurry" vector that doesn't match any single query well.
- Chunk size affects both **retrieval precision** (can we find the right piece?) and **generation quality** (does the LLM get enough surrounding context to answer fully?).

How does it work?

Common strategies:

- **Fixed-size chunking** — split every N tokens/characters, often with some overlap between chunks so context isn't lost at chunk boundaries
- **Recursive/structural chunking** — split along natural document structure first (sections → paragraphs → sentences), falling back to fixed-size only when a piece is still too large
- **Semantic chunking** — split where the topic actually changes (using embedding similarity between adjacent sentences to detect topic shifts)
- **Chunk overlap** — including the last N tokens of the previous chunk at the start of the next, so a sentence spanning a chunk boundary isn't orphaned
- **Contextual/enriched chunking** — attaching metadata or a summary to each chunk (e.g., section header prefix, parent-document summary) so the chunk retains context even when retrieved in isolation (see your Q18, Q27)

Important terminology

- **Chunk size** — the target length of each chunk (in tokens or characters)
- **Overlap** — shared text between consecutive chunks
- **Parent-document retrieval** — a pattern where you retrieve small precise chunks but pass the larger parent section/document (or a summary of it) to the LLM for generation
- **Chunk metadata** — additional fields (page number, section, document title) stored alongside a chunk to aid filtering or context

Databricks implementation

- Chunking is typically done in a Databricks notebook/pipeline using Python (e.g., LangChain's text splitters, or custom PySpark UDFs) as part of a **Delta Live Tables** or batch Spark pipeline
- Chunks are written to a **Delta table in Unity Catalog** — each row = one chunk, with a **unique identifier per row** and the original filename/metadata as columns
- **Change Data Feed (CDF) must be enabled** on this Delta table if it will back a Mosaic AI Vector Search Delta Sync Index

Real-world example

A 300-page equipment manual is split by section headers first (recursive chunking), and each resulting chunk is prefixed with its section title (e.g., "Section 4.2: Troubleshooting Power Issues — [chunk text]") so that even retrieved in isolation, the LLM knows what topic the chunk covers.

Simple example

```python
# conceptual example, not exact API
chunks = recursive_character_splitter(
    document_text,
    chunk_size=500,       # tokens
    chunk_overlap=50      # tokens shared between consecutive chunks
)
for i, chunk in enumerate(chunks):
    write_row(chunk_id=f"{doc_id}_{i}", text=chunk, source_doc=doc_id)
```

Architecture/workflow

```
[Raw Document] → [Extract text (Python: pytesseract for scanned images,
                   beautifulsoup for HTML, PyPDF-style libs for text PDFs)]
               → [Chunk (fixed-size / recursive / semantic)]
               → [Enrich (headers, summaries, metadata)]
               → [Write to Delta table with unique ID per row]
               → [Enable CDF on the table]
               → [Sync to Vector Search Index]
```

Important exam points

- **Chunk size trade-off:** too small → loses context, incomplete answers; too large → dilutes semantic focus, may exceed embedding model context length, increases token cost.
- **Reducing chunk record count to fit a storage limit** (a real official sample question pattern): increasing chunk size AND decreasing chunk overlap both reduce the total number of chunks/records produced from the same corpus.
- **Document content extraction package selection** depends on source format: `pytesseract` for OCR on scanned images (`.jpeg`/`.png`), `beautifulsoup`/`scrapy`/`pyquery` for HTML/web content, PDF-text libraries for native-text PDFs. Pick based on **format**, not on habit.
- Choosing/tuning chunking strategy should be done **empirically**, using retrieval evaluation metrics (recall, NDCG) or an LLM-as-judge against known question/answer pairs — not by intuition (this is directly tested — see your Q3).

Common mistakes/traps

- Assuming a bigger chunk is always "more context = better" — bigger chunks can dilute embedding specificity and increase noise in the prompt.

- Confusing "reduce chunk overlap" with "reduce chunk size" — they have different effects: reducing overlap primarily reduces redundant record count; reducing chunk size increases the number of chunks (opposite direction!).
- Using the same chunking strategy for every document type regardless of structure (a technical manual with headers should be chunked differently than a chat transcript).

Comparison with related concepts

	Small chunks	**Large chunks**
Retrieval precision	Higher (focused)	Lower (may match on irrelevant portions)
Context completeness	Lower (may miss surrounding info)	Higher
Number of chunks needed for full context	More	Fewer
Storage/index size	Larger (more rows)	Smaller

Example exam questions (style)

- "A RAG system's answers are often incomplete because the retriever only fetches short document fragments missing surrounding necessary detail. What are two things to try?" → Increase chunk size; add section-header prefixes/metadata (matches your Q18)
- "1.5 million embeddings need to fit in a vector database limited to 1 million. What TWO actions reduce record count?" → Increase chunk size; decrease chunk overlap

---

Topic 3: Embeddings and Embedding Models

What is it?

An embedding is a numerical vector (a list of numbers, e.g., 768 or 1536 dimensions) that represents the *meaning* of a piece of text, such that texts with similar meaning have vectors that are close together in that vector space (measured by cosine similarity or another distance metric).

Why is it used?

Embeddings are what make **semantic search** possible — finding documents that mean the same thing as a query even if they don't share the same exact words. This is fundamentally different from keyword search, which only matches literal text overlap.

How does it work?

1. An embedding model (a specialized, usually smaller, neural network) takes text as input and outputs a fixed-length vector.
2. Similar meanings → similar vectors → small distance/high cosine similarity between them.
3. To search: embed the query the same way, then find the stored document vectors closest to the query vector.

Important terminology

- **Embedding dimension** — the length of the output vector (e.g., 384, 768, 1536, 4096). Larger isn't automatically better — it's a cost/accuracy tradeoff (larger dimension = more storage, more compute, sometimes but not always more accurate).
- **Embedding model context length** — the maximum input length the embedding model itself accepts (distinct from the LLM's context length!). Your chunk size must be ⩽ this.
- **Cosine similarity** — the standard metric for comparing embedding vectors (measures the angle between two vectors, not their magnitude)
- **MTEB (Massive Text Embedding Benchmark)** — a public leaderboard ranking embedding models across many general tasks; useful as a signal but **not a substitute for domain-specific evaluation** (a recurring exam trap — see your Q35)
- **Self-managed / precomputed embeddings** — embeddings you compute yourself outside of Databricks Vector Search and store directly, vs. letting Vector Search compute them automatically from raw text

Databricks implementation

- Embedding models are available via **Foundation Model APIs** (Databricks-hosted models like `databricks-gte-large-en`, or external models via External Models)
- Mosaic AI Vector Search can either (a) compute embeddings for you automatically from a text column, or (b) accept **precomputed/self-managed embedding vectors** you supply yourself
- Regardless of which mode you use, the **source Delta table still needs Change Data Feed enabled** for a Delta Sync Index (a commonly tested nuance — see your Q22)

Real-world example

Searching "How do I reset my password?" should also match a document chunk that says "Steps to recover account access" — very different wording, same meaning — because their embeddings will be close together even though there's no keyword overlap.

Simple example

```
Text A: "The cat sat on the mat" → embedding vector [0.12, -0.44, 0.91, ...]
Text B: "A feline rested on the rug" → embedding vector [0.14, -0.41, 0.88, ...]
(vectors are close → high cosine similarity → considered semantically similar)

Text C: "Stock prices fell today" → embedding vector [-0.77, 0.33, 0.02, ...]
(vector is far from A and B → low similarity)
```

Important exam points

- **Choosing an embedding model:** prioritize domain relevance to your actual corpus/queries over generic leaderboard (MTEB) rank, and over "most recent" or "multilingual" if those attributes aren't actually needed for your use case.
- **Embedding model context length selection** is its own tested objective, separate from "which model": you must match context length, model size, and embedding dimension to your chunking strategy and your cost/latency/quality priorities. If cost/latency matter more than quality and chunks are ⩽512 tokens, pick the *smallest* model/context length that still covers your chunk size — don't over-provision.
- Precomputed embeddings are fully supported in Vector Search — don't assume "self-managed" is unsupported or second-class.

Common mistakes/traps

- Assuming the embedding model with the highest MTEB score is always the right choice — domain fit matters more for your specific retrieval task.
- Choosing a much larger context-length/embedding-dimension model than your chunk size requires, when cost/latency were explicitly prioritized — over-provisioning wastes money and adds latency for no benefit.
- Forgetting embeddings for the **query** must come from the **same embedding model** used to embed the documents — mixing models breaks the vector space comparison.

Comparison with related concepts

	Embedding model	Generation LLM
Output	A vector (numbers)	Text
Purpose	Represent meaning for search/comparison	Produce a human-readable response
Typical size	Smaller, specialized	Larger, general-purpose
Used in RAG for	Retrieval step	Generation step

Example exam questions (style)

- "A team wants to experiment with different embedding models to improve RAG performance. What should guide their choice?" → Domain relevance of training data, not just leaderboard rank.
- "Chunks are capped at 512 tokens; cost and latency matter more than quality. Which embedding config best fits?" → The smallest model whose context length still covers 512 tokens (don't over-provision a 32K-context, 14GB model for 512-token chunks).

Topic 4: Mosaic AI Vector Search

What is it?

Mosaic AI Vector Search is Databricks' managed vector database service — it stores embedding vectors, keeps them in sync with a source Delta table, and lets you run fast approximate-nearest-neighbor (ANN) similarity searches against them.

Why is it used?

Storing and searching millions (or billions) of high-dimensional vectors efficiently requires specialized indexing (you can't just linearly scan every vector for every query at scale) — this is what Vector Search's ANN indexing solves, while also handling governance (Unity Catalog integration), incremental sync, and scaling.

How does it work?

- You create an **index** backed by a **source Delta table** (containing your chunk text + optionally precomputed embeddings + metadata)
- Vector Search either computes embeddings itself (given a specified embedding model) or ingests your precomputed embeddings
- The index continuously or periodically **syncs** with changes to the source table (insertions, updates, deletions)

- At query time, you supply a query vector (or query text, which gets embedded automatically) and get back the top-K most similar rows

Important terminology

- **Delta Sync Index** — an index kept automatically in sync with a Delta table via Change Data Feed; requires CDF enabled on the source table
- **Direct Vector Access Index** — an index you manage more directly via API calls, without automatic Delta table syncing (useful for non-Delta or highly custom workflows)
- **Sync modes: Continuous** (near real-time sync — higher cost, freshest data) vs. **Triggered** (sync only when manually/periodically triggered — lower cost, appropriate for infrequently-changing data)
- **Standard vs. Storage-optimized Vector Search:** Storage-optimized is designed for much larger-scale indexes (hundreds of millions+ of vectors) at lower cost per vector, potentially with some latency/feature tradeoffs vs. Standard
- **Metadata filtering** — filtering search results by structured fields (e.g., date range, category) alongside the vector similarity search
- **Hybrid search** — combining keyword (lexical) search with vector (semantic) search to catch both exact-term matches and semantic matches
- **Reranking** — a second-pass model that reorders an initial set of retrieved candidates by more precise relevance (see Topic 5)

Databricks implementation

```
1. Prepare source Delta table (chunked text, unique ID column, metadata columns)
2. Enable Change Data Feed on that table (required for Delta Sync Index)
3. Create Vector Search endpoint (compute layer)
4. Create index on the endpoint, pointing at the source table:
   - choose embedding source (computed vs. precomputed column)
   - choose sync mode (continuous vs. triggered)
   - choose index type (standard vs. storage-optimized) based on scale
5. Query the index (via SQL function, Python SDK, or REST API), optionally with:
   - metadata filters
   - hybrid search enabled
   - reranking enabled
```

Real-world example

An e-commerce marketplace with 500 million product listings uses storage-optimized Vector Search (for scale) with metadata filtering (to support "preset filters" like brand/price range)

and hybrid search (so exact product-name searches still work well alongside semantic "comfortable running shoes" style queries).

Simple example

```sql
-- conceptual SQL-style query against a Vector Search index
SELECT * FROM VECTOR_SEARCH(
  index => 'catalog.schema.product_index',
  query_text => 'comfortable running shoes for wide feet',
  num_results => 10,
  filters => "category = 'footwear'"
)
```

Architecture/workflow

```
[Delta Table: chunks + metadata] --(CDF)--> [Delta Sync Index] --(embed if
needed)--> [ANN Index]

↑                                                                    [Query text/vector +
filters] --┘

↓                                                                    [Top-K
results, optionally reranked]
```

Important exam points

- **Failure mode to memorize:** Delta Sync Index creation fails most commonly because **Change Data Feed is not enabled** on the source table — this applies whether you're using computed or precomputed embeddings (see your Q22).
- **Scale drives index type:** very large corpora (hundreds of millions of rows) call for storage-optimized indexing; standard vector search doesn't scale to that volume as cost-effectively.
- **Sync mode choice:** infrequent updates → Triggered sync (lower cost); frequently changing / freshness-critical data → Continuous sync.
- **Evaluating ANN algorithm choices (e.g., HNSW vs. LSH) for semantic accuracy:** compare cosine similarity of the embeddings of returned results — not BLEU, ROUGE, or

Levenshtein distance, which are text-generation/string metrics, not embedding-space metrics.

Common mistakes/traps

- Assuming precomputed embeddings mean you can skip CDF — you can't; CDF is about tracking row-level changes to the *table*, unrelated to who computed the embedding.
- Choosing Standard vector search "by default" without checking scale requirements against Storage-optimized's intended use case.
- Applying BLEU/ROUGE to evaluate retrieval quality — these are text-overlap metrics for generated text, not for embedding similarity.

Comparison with related concepts

	Delta Sync Index	**Direct Vector Access Index**
Source	Delta table (auto-synced via CDF)	Managed directly via API
Best for	Standard RAG pipelines already using Delta tables	Custom/non-Delta workflows, more manual control

	Continuous sync	**Triggered sync**
Freshness	Near real-time	Only on manual/scheduled trigger
Cost	Higher	Lower
Best for	Frequently changing data	Rarely changing / batch-updated data

Example exam questions (style)

- "A Vector Search index creation fails when using precomputed embeddings on a Delta table source. What's the most likely cause?" → Change Data Feed not enabled on the source table.
- "A team needs the highest scale (500M+ listings), preset metadata filters, and best possible match quality with low update frequency. What configuration fits?" → Storage-optimized index + metadata filtering + hybrid search + an appropriately-chosen sync mode for the stated update cadence.

Topic 5: Retrieval Optimization — Reranking & Retrieval Evaluation

What is it?

Retrieval is the step where, given a query, you find the most relevant chunks from your index. Reranking is an optional *second pass* that reorders an initial batch of retrieved candidates using a more precise (often more computationally expensive) relevance model, since the fast initial ANN search sometimes ranks true relevance imperfectly.

Why is it used?

Initial vector similarity search is fast but approximate — it can occasionally rank a less-relevant chunk above a more-relevant one. A reranker, applied to only the top-K candidates (a much smaller set), can afford to use a more accurate (often cross-encoder) relevance model to fix the ordering before you pass results to the LLM.

How does it work?

1. Vector search retrieves an initial candidate set (e.g., top 20)
2. A reranker model scores each candidate against the query with a more precise/expensive method
3. The reordered top-K (e.g., top 5) is what actually gets passed into the LLM prompt

Important terminology

- **Cross-encoder** — a model architecture commonly used for reranking, which jointly processes the query and candidate together (more accurate but slower than the embedding-based approach used for the initial retrieval)
- **Recall** — of all truly relevant chunks that exist, what fraction did we retrieve?
- **NDCG (Normalized Discounted Cumulative Gain)** — a ranking-quality metric that rewards placing the most relevant results higher in the returned list, not just retrieving them at all

Databricks implementation

- Mosaic AI Vector Search supports **built-in reranking** as a query-time option
- Retrieval evaluation is typically done via **MLflow's built-in evaluation metrics** for retrieval (e.g., recall, NDCG) inside `mlflow.genai.evaluate()`, using a curated dataset of queries with known-relevant chunk labels

Real-world example

A support search returns five documents; the three highest-ranked by initial vector similarity turn out (per evaluation) to be less relevant than the two ranked lower. Enabling a reranker

step reorders results so the two truly most relevant documents surface at the top before being sent to the LLM.

Simple example

```
Initial retrieval (by cosine similarity): [Doc C, Doc A, Doc E, Doc B, Doc D]
After reranking (by cross-encoder relevance): [Doc B, Doc D, Doc C, Doc A, Doc E]
→ Pass only top 3 post-rerank to the LLM: [Doc B, Doc D, Doc C]
```

Important exam points

- When retrieved documents are ranked out of order relative to true relevance, the textbook fix is **adding a reranker**, not fine-tuning the embedding model or manually reordering via prompt instructions.
- Retrieval evaluation should be **quantitative and systematic** (recall/NDCG on a labeled dataset, or LLM-as-judge scoring), not based on manual spot-checking or intuition.

Common mistakes/traps

- Trying to fix a ranking problem by prompting the LLM to "put the most relevant documents first" — unreliable, and treats a retrieval-layer problem as a generation-layer patch.
- Confusing recall (did we find it at all) with ranking quality (NDCG) — a system can have perfect recall but still rank the right answer last.

Comparison with related concepts

	Initial vector search	**Reranking**
Speed	Fast (ANN, approximate)	Slower (applied to fewer candidates)
Accuracy	Good but approximate	Higher precision on the smaller candidate set
When applied	First pass, over the whole index	Second pass, over top-K candidates only

Example exam questions (style)

- "Search results have five docs; the top 3 by relevance score are ranked below two others that are more relevant per evaluation. What should the team do?" → Add a reranker to reorder by true relevance.

Quick self-check before moving on

You should now be able to answer, without looking back:

1. What's the difference between what RAG changes and what fine-tuning changes?
2. Name two chunking strategies and when you'd pick each.
3. Why does precomputed embeddings still require Change Data Feed on the source table?
4. When would you choose Storage-optimized Vector Search over Standard?
5. What's the difference between recall and NDCG?
6. Why isn't "highest MTEB score" always the right way to pick an embedding model?

If any of these feel shaky, re-read that section before moving to Part 2.

Next: Study Material Part 2 — Agentic AI, Agent Bricks, MCP Servers & Multi-Agent Systems (Sections 1 & 3-4's agent-related objectives — the second-largest tested cluster).